

Lab 3-2：前后端分离实践 —— 构建智能出行规划器

课程：生成式软件开发 工具：Trae（推荐使用 Builder / Workspace 模式） 提交截止：2026.04.31 23:59 提交方式：eLearning-assignment

一、实验目标

通过本次实验，你将：

1. **理解** 前后端分离的基本开发方式，明确前端、后端与第三方 API 的职责边界
2. **实践** 将 Lab 3-1 中已经申请和使用过的 API 继续整合进一个更完整的软件系统
3. **构建** 一个具有真实使用价值的“智能出行规划器”——支持地点管理、行程安排、天气信息展示、规划管理与查看
4. **体验** 用 Trae 协助完成一个多模块、多文件、前后端协作的小型项目开发过程

二、课前阅读

在开始本实验前，请先阅读知识文档：

- [lab3_02_knowledge_v1.md](#)

这份知识文档会帮助你理解：

- Lab 3-1 为什么不适合继续沿用纯前端直连大模型 API 的方式
- 什么是前后端分离，以及为什么这一实验需要引入后端
- Skills 是什么，以及本实验推荐如何使用
- 为什么本实验默认推荐使用 Vue 3 + Python + FastAPI + SQLite

三、实验任务

3.1 系统概述

本实验要求你实现一个**前后端分离的智能出行规划器**。这个系统面向有短途出行、一日游或周末出行需求的用户，帮助用户完成从“创建规划”到“查看建议”的完整流程。

从实验任务出发，这个系统至少要解决下面几个问题：

- 用户如何创建一份新的出行规划，并填写日期、预算、人数、偏好等基本信息
- 用户如何在地图上选择地点，并把这些地点加入当前规划
- 用户如何根据地点和天气安排当天的行程顺序
- 用户如何保存这份规划，并在之后继续查看或编辑
- 系统如何在最后基于已有规划内容生成一段 AI 辅助建议

因此，这个系统的核心不是“聊天”，而是**围绕一份出行规划组织数据、页面和交互流程**。
你最终要交付的是一个能够完成以下完整流程的 Web 系统：

1. 用户进入系统后，可以创建或打开一份出行规划
2. 用户填写规划基本信息，例如日期、预算、人数、偏好
3. 用户通过地图选择地点，并将地点加入当前规划
4. 系统根据地点和时间查询并展示天气信息
5. 用户按上午 / 下午 / 晚上等时间段组织行程
6. 用户可以保存这份规划，并在之后再次打开继续编辑
7. 系统基于地点、天气、时间安排和预算信息生成 AI 辅助总结

从系统结构上看，这个实验至少包含以下几个核心部分：

1. 规划管理模块

负责创建、查看、编辑和保存出行规划。
这是整个系统的数据中心，其他模块都围绕“当前规划”展开。

STEP 1

规划信息

新建规划

规划标题

雨天室内优先

出行日期

2026/05/10

预算 (元)

360

人数

3

偏好

希望节奏轻一点，优先室内活动，避免淋雨

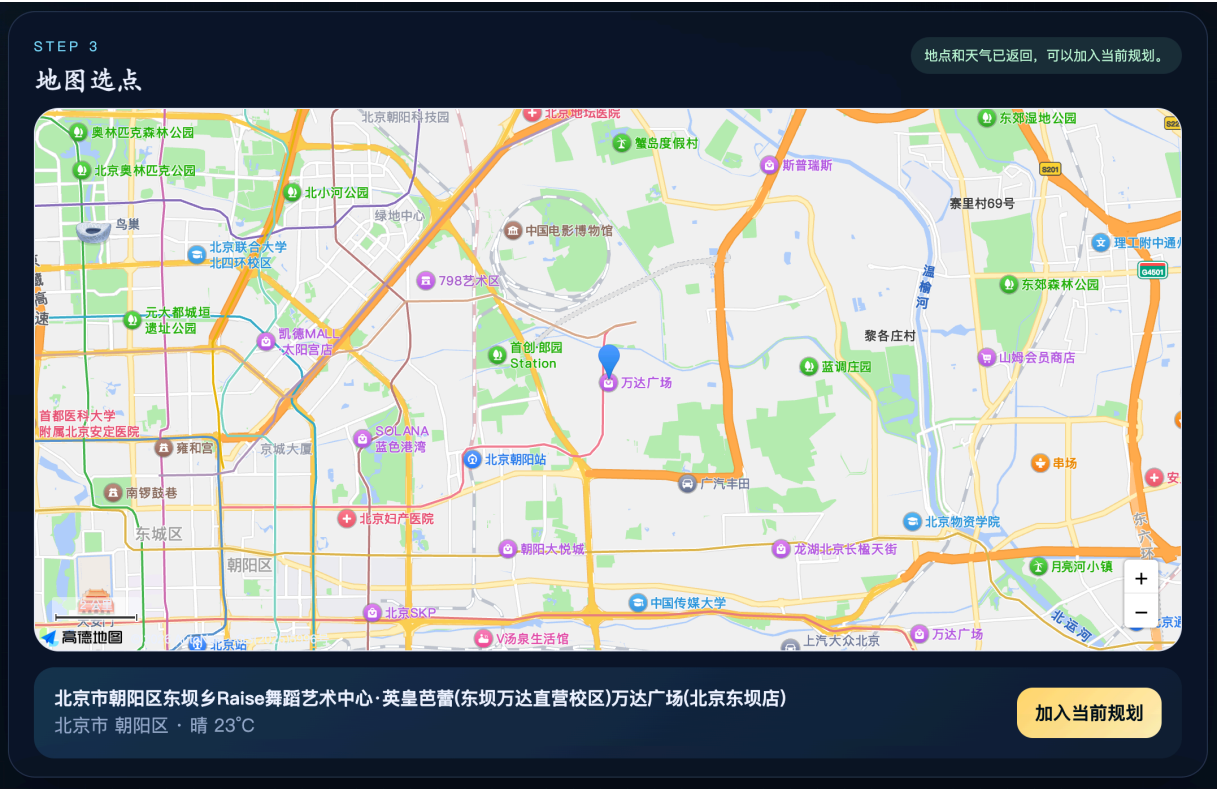
备注

这份示例用于展示规则检查和 AI 风险提示。

2. 地点管理模块

负责地图选点、地点加入规划、地点查看与调整。

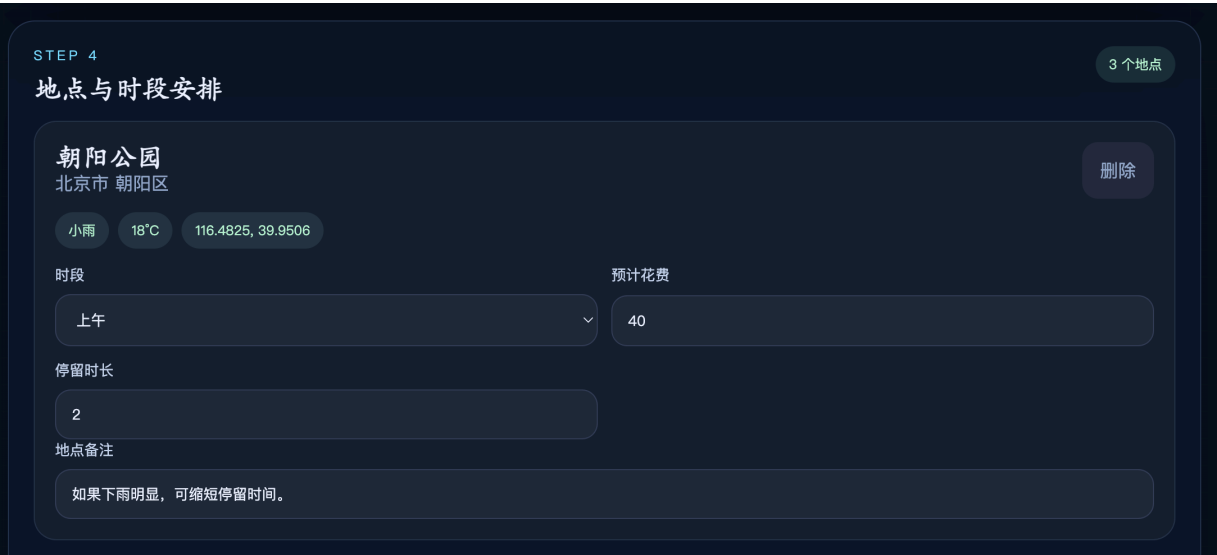
用户选择的地点不是临时展示，而是规划数据中的正式组成部分。



3. 天气展示模块

负责根据规划中的地点和时间获取天气信息，并把天气结果展示在前端页面中。

天气信息不是独立存在的，而是服务于用户的出行安排。



4. 行程安排模块

负责把地点组织到不同时间段中，例如上午、下午、晚上。

这一部分要体现出“规划”的含义，而不是只保存一个地点列表。

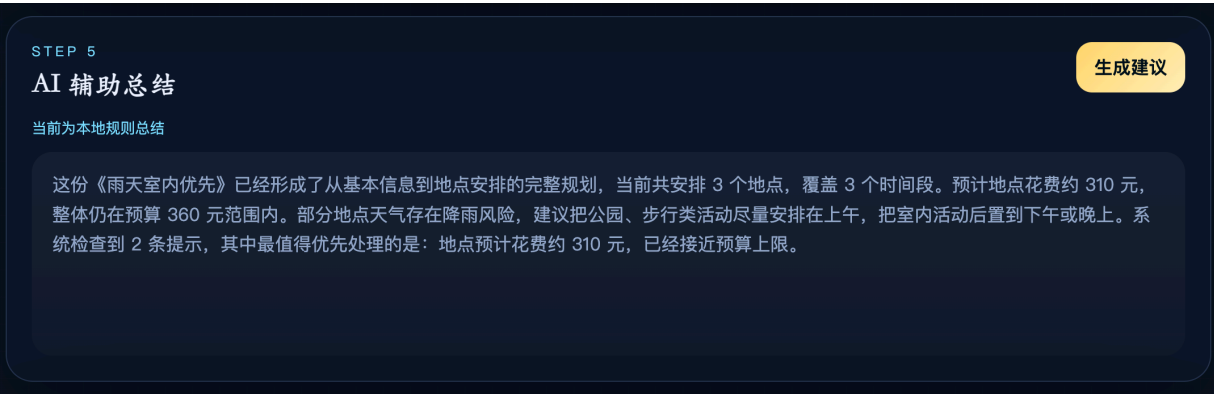
5. AI 辅助总结模块

负责在已有规划、天气和时间安排的基础上，生成一段自然语言建议。
它的作用是帮助用户理解当前规划是否合理，而不是替代前面的软件功能。

为了避免理解偏差，你需要始终记住：

- 这不是一个“地图展示页面”
- 这不是一个“天气查询页面”
- 这不是一个“AI 聊天页面”

它应该是一个以出行规划为主线，把地图、天气和大模型能力整合起来的完整小系统。（下面的图片只是一个非常简单的demo，我们可能需要一个更好的建议～）



3.2 核心任务：构建“智能出行规划器”

默认推荐技术栈：

- 前端：Vue 3
- 后端：Python
- 后端框架：推荐 FastAPI
- 数据存储：推荐 SQLite

说明：

- 本实验默认按以上推荐栈完成，便于统一指导、联调和后续发布
- 如果你确实希望更换技术栈，需要保证功能完整，并能自行解决开发与部署中的适配问题

3.3 基础功能（必做）

模块	功能要求
规划管理	用户可以新建、查看、编辑并保存一份出行规划，填写基本信息，如日期、预算、人数、偏好等

模块	功能要求
地点管理	用户可以在地图上选择地点并加入当前规划，同时支持查看、删除或调整地点信息
天气展示	根据规划中的地点和时间，展示相关天气信息或天气摘要
行程安排	支持按时间段组织规划，例如“上午 / 下午 / 晚上”

3.4 软件实现重点（必做）

本实验的重点不在“让 AI 多说话”，而在于完成一个结构清晰的小系统。请特别注意以下几个方面：

前端部分

- 有清晰的页面或模块划分
- 能正确处理加载中、请求失败、空数据等状态
- 表单和交互流程清楚，用户知道下一步该做什么
- 规划结果展示清晰，不只是把原始 JSON 打在页面上

后端部分

- 能接收前端请求并返回结构化数据
- 能统一处理天气等外部 API 的调用
- 能保存和读取出行规划数据
- 能对明显不合理的规划给出提示或检查结果

前后端协作

- 前后端职责边界清晰
- 不要把所有逻辑都塞进前端
- 不要让前端直接暴露大模型 API Key
- 尽量由后端统一处理第三方服务调用

3.5 AI 能力（必做）

在基础功能完成后，需要加入一个“AI 辅助总结”模块：

- 基于地点、天气、时间安排、预算等信息
- 生成一段自然语言的出行建议
- 说明当前规划的优点、风险和可改进点

示例：

你本次选择的地点以户外活动为主，但下午有小雨，建议将公园安排提前到上午，把室内活动放到下午。整体预算较紧，建议减少跨城区移动。

说明：这一部分属于本实验的正式要求。你需要让 AI 在系统中承担一个明确而合理的辅助角色，但不应让它取代前面的软件功能实现。

3.6 自主拓展（选做）

完成基础要求后，可以从以下方向中任选一项或多项拓展：

- 多日规划：支持 2 天及以上的出行安排
- 历史规划列表：查看、删除、复制已有规划
- 导出功能：导出为 Markdown、JSON 或打印友好的行程单
- 用户偏好设置：保存用户偏好，例如“喜欢自然景观”“不想安排太满”
- 规划模板：快速生成“半日游 / 一日游 / 周末游”模板
- 更丰富的规则检查：例如预算超出提醒、时间安排冲突提醒、雨天户外风险提醒

四、实验要求

4.1 开发方式

- 使用 **Trae** 辅助完成开发
- 采用多轮对话方式与 Trae 协作，每轮聚焦一个子目标
- 建议先完成后端基础能力，再逐步连接前端页面
- 建议至少在开发过程中主动尝试使用一次或多次 **Skills**
- **Lab 3-2** 文档发布后
- 实验课上会统一讲解如何把这部分前后端项目发布到云服务器，以及发布过程中常见的问题与处理步骤

4.2 前后端分离要求

- 必须区分前端与后端目录或工程
- 前端负责交互展示与用户操作
- 后端负责数据处理、业务逻辑和外部服务整合
- 原则上按推荐栈完成：前端 **Vue 3**，后端 **Python**

4.3 数据管理要求

- 规划数据必须支持保存与再次读取

- 默认推荐使用 SQLite 作为持久化方案，便于本地开发与后续部署演示
- 如果你选择其他存储方式，必须能说明你为什么这样设计
- 如果只保存在内存中，程序重启后丢失数据，则不能满足本实验“规划管理”的基本要求

4.4 API Key 与安全

- 第三方 API Key 不得直接写死在公开提交的代码中
- 前端代码中不应直接暴露大模型 API Key
- 建议由后端统一处理大模型调用

提示：这正是前后端分离的一个重要价值：前端只负责调用你自己的后端接口，敏感凭据保存在后端。

4.5 版本管理

- 使用 Git 管理代码
- 建议按阶段提交，例如：
 - feat: 初始化 Vue3 + Python + FastAPI 项目结构
 - feat: 实现规划管理
 - feat: 接入天气信息
 - feat: 完成前端页面整合
 - feat: 添加AI辅助总结

五、提交内容

请提交以下内容（打包为压缩包），命名格式：姓名_学号_lab3-2

1. 项目代码

- 完整的前端代码
- 完整的后端代码
- 必要的运行说明

2. AI 协作记录

- 导出的聊天记录
- 如果使用了 Skills，请在记录中能看出你使用了哪些 Skill

3. 版本管理记录

- git log --oneline 截图

4. 实验总结

- 你的前端和后端分别负责了什么？
- 你是如何组织规划数据的？

- 你如何处理天气信息和规划之间的关系？
- AI 在系统中的角色是什么，为什么它不是主角？
- 在这个实验中，你觉得最难的是前端、后端、数据管理，还是前后端协作？为什么？

六、评分标准

基础评分（满分 100 分）

维度	占比	说明
基础功能	50%	前后端分离、规划管理、地点管理、天气整合、行程安排等核心功能完成度
工程实现质量	10%	前后端职责清晰，结构合理，状态处理和数据管理较完整
实验总结	20%	对系统设计、数据管理、前后端协作和 AI 角色有清晰反思
AI 协作与版本管理	10%	Trae 协作记录完整，版本提交规范
拓展功能	5%	在基础功能之外实现了有价值的扩展
Skills 使用	5%	能结合任务主动使用合适的 Skills，并体现出实际帮助

说明：

- 基础功能 是本实验最核心的评分部分
- 拓展功能 分值不高，但鼓励你在完成主线后继续完善产品
- Skills 使用 不看“用了几个”，更看你是否在合适的时候用对了工具

七、提示与建议

7.1 推荐开发顺序

1. 先按推荐栈搭建 Vue 3 + Python 的工程结构
- ↓
2. 先完成后端最基本的数据读写能力
- ↓
3. 再完成地图选点与前端展示
- ↓
4. 接入天气信息并展示在规划页面

- ↓
5. 加入时间段安排与规划编辑能力
- ↓
6. 最后再加入 AI 辅助总结

7.2 关于系统边界

本实验最容易出现的问题是：

- 前端写得很多，但后端只是“转发一下”
- 有天气 API 和大模型 API，但没有真正形成“规划系统”
- 页面能展示数据，但没有“保存、查看、继续编辑”的闭环

因此你需要不断提醒自己：你在做的是个出行规划器，而不是一个天气聊天页面。

7.3 关于功能取舍

请优先保证以下能力真的可用：

- 能创建规划
- 能加入地点
- 能安排时间
- 能保存和读取
- 能看到天气信息
- 能生成 AI 辅助总结

不要一开始就追求：

- 很炫的 UI
- 很复杂的 AI 自动规划
- 很多但不稳定的高级功能

7.4 关于推荐技术栈

本实验默认推荐使用：

- 前端：Vue 3
- 后端：Python
- 后端框架：FastAPI
- 存储：SQLite

这样做的好处是：

- 助教在实验课上更容易统一讲解
- 同学之间更容易互相交流和排查问题

- 后续讲解云服务器发布时，步骤可以尽量统一

7.5 关于 AI 的角色

在本实验里，AI 更适合做这些事情：

- 总结一份规划
- 解释为什么某些安排可能不合理
- 给出自然语言建议

而不适合替代这些事情：

- 基础的数据结构设计
- 前后端的职责划分
- 保存和读取规划数据
- 前端页面状态管理

7.6 常见问题

问题	可能原因	建议
前后端都能跑，但功能很乱	没有先设计好数据结构和页面流程	先画出规划对象包含哪些字段，再开始写代码
地图、天气、规划数据互相脱节	没有把“地点”作为核心对象统一管理	先定义地点数据结构，再让其他功能围绕它展开
保存功能做不出来	一开始只考虑页面展示，没有考虑存储模型	先确定数据存哪里、怎么读取，再做界面
AI 功能很强，但整体系统很弱	把太多精力放在 prompt 上	先完成软件主体，再做 AI 增强
发布时环境不一致	本地开发栈和部署环境差异太大	尽量按推荐栈完成，减少课上发布时的额外适配成本

如有疑问，请在课程群中提问，或在实验课时向助教咨询。